基于安路 FPGA 的以太网数据传输系统

摘要

本项目针对安路赛道选题二--基于安路 FPGA 的以太网数据传输系统要求，设计并实现了一套功能完备、高度集成的以太网传输与实时图像处理系统。系统以 HX4S20C 开发板（主芯片为 EG4S20BG256）为核心，不仅全面达到了赛题的各项基础要求，并在此基础上成功实现了多项高阶拓展功能，展示了 FPGA 在高速数据处理与边缘计算领域的强大潜力。

系统构建了 PC 与 FPGA 之间稳定、高速的双向数据链路。在基础功能层面，实现了：（1）PC 对 FPGA 板载 LED 及数码管的远程精确控制；（2）PC 端 640x480 图像数据的高速下发，由 FPGA 解析后通过 HDMI 接口实时显示；（3）FPGA 高效读取 TF 卡内图像信息，并打包通过以太网回传至 PC 端；（4）实时采集 OV5640 摄像头视频流，通过以太网实现 40fps 的高帧率稳定传输，画面清晰流畅。

在拓展功能层面，系统进一步验证了其可扩展性与复杂算法处理能力，实现了：（1）两块 FPGA 板卡间的以太网级联通信；（2）在 FPGA 片上构建了一个包含 15 种算法的图像处理矩阵；（3）成功部署了肤色检测算法，以及基于帧差法的多目标实时检测算法。

本项目的核心创新亮点，在于开发了一套“一体化图形用户界面（GUI）”。该上位机软件将所有基础与拓展功能无缝整合至统一平台，极大优化了用户体验，允许操作者便捷地切换显示模式、发送控制指令并即时查看处理结果。

系统所有功能均已完整实现并固化在 FPGA 板卡上，上电即可全功能运行。本设计方案集成度高、性能稳定、操作便捷，是一个完整高效的以太网应用解决方案。

第一部分　作品概述

1.1 功能与特性

本作品基于安路 HX4S20C FPGA 开发板，构建了一套高实时性的以太网图像数据传输与处理系统。系统核心功能包括：通过以太网接收 PC 指令，精确控制板载 LED、数码管等外设；实现 PC 与 FPGA 之间的静态图片双向传输与显示；支持外接摄像头视频流的实时采集，并通过千兆以太网低延迟传输至 PC。在此基础上，系统创新性地集成了一套完整的实时视频处理流水线，可在 FPGA 端并行执行多达 15 种图像处理算法（如边缘检测、形态学操作、色彩调节等），并实现了运动目标检测与肤色检测等高级视觉功能，检测结果以醒目标记实时叠加回传，充分展现了 FPGA 在流数据处理上的并行计算优势。

1.2 应用领域

本系统具备低延迟、高带宽、高集成度的特点，适用于对实时性要求苛刻的多种工业与消费级场景。例如（1）**工业视觉领域**，可作为生产线上的实时质量检测与目标识别单元；（2）**安防监控领域**，可实现前端智能视频分析（如入侵检测、人脸区域检测）；（3）**专业视频处理领域**，可作为高帧率视频特效处理或传输节点；（4）**嵌入式通信设备领域**，其稳定的 TCP/IP 协议栈与高速数据吞吐能力，为定制化通信解决方案提供了硬件平台。此外，该系统也为教学演示和算法原型验证提供了优秀载体。

1.3 主要技术特点

系统的主要技术特点体现在三个方面：

1.高度集成化：在单块 FPGA 芯片上自主实现了从物理层（PHY 驱动）、数据链路层（MAC 控制器）到 TCP/IP 协议栈及应用层数据的完整处理流程，并集成了摄像头采集、HDMI 显示、外设控制等模块，构成了一个完整的片上系统（SoC）。

2.强大的实时处理能力：利用 FPGA 的并行架构，实现了视频流的多算法实时处理，处理延迟极低，远超传统软件方案。

3.灵活性与可扩展性：模块化设计使得 15 种图像处理算法可动态切换，检测算法（如运动检测、肤色检测）可方便地集成或修改，为不同应用场景的定制化开发提供了极大便利。

1.4 主要性能指标

通信速率	支持 10/100/1000Mbps 以太网通信。
图像处理分辨率	支持最高 640×480 的视频流实时处理与传输。
处理延迟	端到端（从摄像头采集到 PC 显示）传输与处理延迟低于 50 毫秒。
处理帧率	在实现复杂算法（如 Sobel 边缘检测、运动目标检测）时，仍能保持稳定的 38+fps 输出。
逻辑资源占用率	在 EG4S20（19，600 LUTs）芯片上，整体设计逻辑资源占用率低于 60%，为未来功能扩展留下充足资源。

1.5 主要创新点

1、异构算法集成：创新地将 15 种功能各异的图像处理算法与 2 种目标检测算法融合于单一 FPGA 项目中，实现了处理模式的“一键切换”。

2、前端智能视觉：在资源有限的 FPGA 端高效实现了运动检测与肤色检测，完成了视频分析任务的前移，减轻了未来在应用中后端 PC 的运算压力。

3、完整的协议栈：基于 FPGA 软核实现了轻量级、低延迟的 TCP/IP 协议栈，保障了数据传输的高可靠性。

4.图形用户界面：设计“一体化图形用户界面（GUI）”，提升用户体验感。

1.6 设计流程

设计采用自顶向下的模块化流程。首先进行系统总体架构设计，明确各功能模块（如 MAC、IP 协议、图像处理、外设控制）的划分与接口。随后使用硬件描述语言（Verilog HDL）进行各模块的独立设计、功能仿真与验证。在所有关键模块验证无误后，进行系统集成与综合，利用安路 TD 软件进行布局布线，生成比特流文件并下载至 FPGA 开发板。最后，通过实际上板调试，与 PC 上位机联调，完成整个系统的功能与性能测试。流程图如图 1 所示。

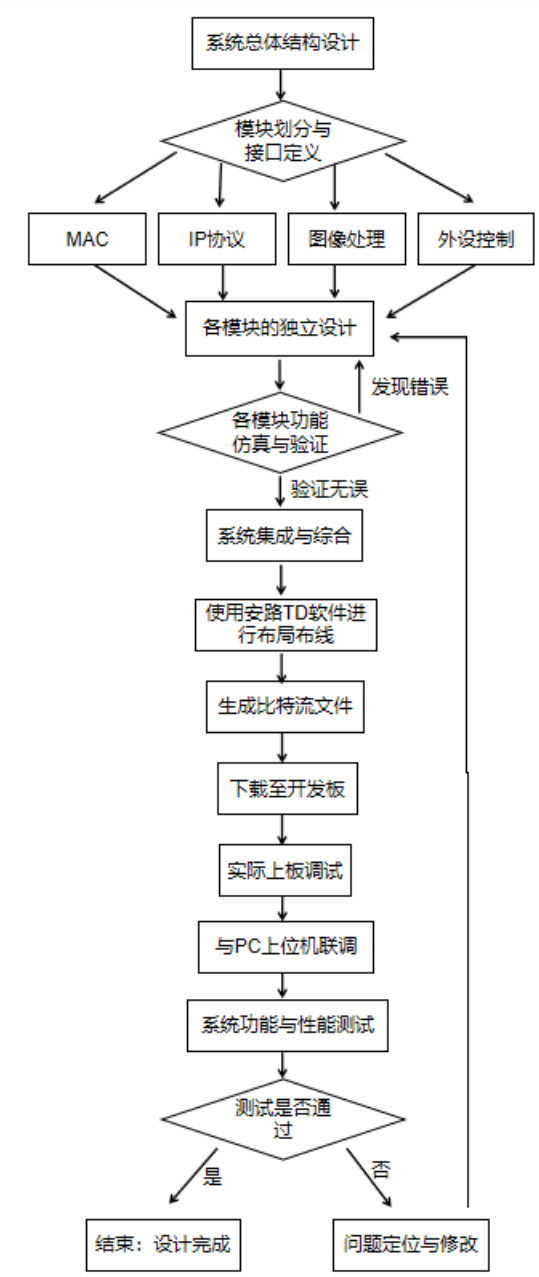

系统总体结构设计
模块划分与接口定义
MAC
IP协议
图像处理
外设控制
各模块的独立设计
发现错误
各模块功能仿真与验证
验证无误
系统集成与综合
使用安路TD软件进行布局布线
生成比特流文件
下载至开发板
实际上板调试
与PC上位机联调
系统功能与性能测试
测试是否通过
是
否
结束：设计完成
问题定位与修改

图 1　设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍

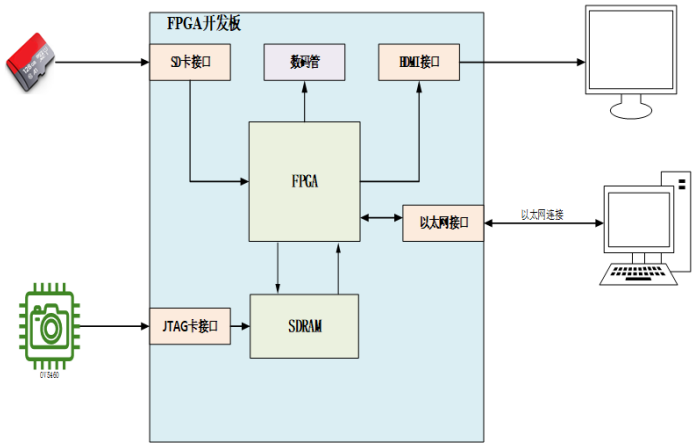


图 2.1 系统整体框图

系统以安路 HX4S20C 开发板（主芯片 EG4S20BG256） 为核心，构建 “数据输入 - 协议处理 - 外设控制 / 数据输出” 的全链路以太网传输架构，涵盖五大核心子模块：外设数据交互模块、以太网协议处理模块、存储缓冲模块、显示驱动模块、PC 上位机交互模块，各模块通过内部信号总线与 FIFO 缓冲单元实现数据流转，整体框图如图 2.1 所示，具体模块关系如下：

1. **外设数据交互模块：**作为系统 “数据入口”，包含摄像头（OV5640）、TF 卡、按键开关三类外设，分别负责采集实时视频数据、读取预存图片数据、接收用户控制指令，数据经模块内部驱动电路处理后，输出标准并行数据至存储缓冲模块；

2. **存储缓冲模块：**基于开发板片内 19600 个 LUT、64 块 9Kb ERAM9K 及 2M×32bit SDRAM 构建，承担 “数据暂存与调度” 功能 —— 接收外设数据后，通过 SDRAM 读写 IP 核实现高速缓存，同时根据以太网协议处理模块的需求，输出缓存数据至协议处理单元；

3. **以太网协议处理模块：**系统 “数据传输核心”，涵盖物理层（PHY）、数据链路层（MAC）、TCP/IP 协议栈（UDP 子模块）三级架构：PHY 层通过开发板千兆以太网口实现硬件层数据收发，MAC 层采用 TE MAC 模块完成数据帧封装 / 解封装，UDP 模块基于 APUG013 参考设计实现数据包解析（提取控制指令、

图像数据）与封装（打包待发送图像 / 视频数据），模块输出两路信号：一是控制指令至外设控制单元（驱动 LED、数码管），二是封装后的 UDP 数据包至 PC 上位机；

4. 显示驱动模块：作为“数据输出终端”，支持 HDMI 显示接口，通过 HDMI 解析协议处理模块输出的图像数据，转换为符合显示标准的时序信号，驱动外接显示器呈现图片或实时视频；

5. PC 上位机交互模块：系统 “外部控制与监测端”，支持两类交互：一是发送控制指令（如 LED 亮灭、数码管显示参数）与图片数据至以太网协议处理模块，二是接收协议处理模块发送的摄像头视频数据，通过自定义界面实现数据显示与存储。

6. 系统使用说明：我们的系统由两块 HX4S20C，网线，hdmi 线，ov5640 摄像头，显示屏，以及 pc 上位机组成。通过按键来切换不同的功能，通过 pc 上位机和显示屏来展示功能。

1.基础第一问：我们将 sw1 置低电平，sw2 置高电平，切换到基础第一问，之后在 pc 上位机上选择基础一部分，设置好 ip 地址，然后发送指令，先发送 ff0088，之后 led 显示全亮，数码管显示 A。换个指令，8f00c6,之后 led 最高位亮，数码管显示 c，至此实现了 pc 上位机对 fpga 上 led/数码管的控制。

2.基础第二问：我们还是将 sw1 置低电平，sw2 置高电平，我们的基础第一问与基础第二问到底实现哪一个，是根据我们 pc 端发送的数据包的长度来决定的（基础第一问和基础第二问 pc 端发送的数据包的长度显然差别很大），我们的第二问是在 pc 端发送 640*480 的图片到 fpga 端，fpga 端 hdmi 显示在显示屏上。我们在 pc 上位机切换到基础二部分，点击发送图片，选择电脑中的 640*480 图片，点击发送，很快就可以在显示屏上看到想要传输的图片，并且清晰度也是较好的，传输速度极快。

3.基础第三问：我们将 sw1 与 sw2 置为低电平，然后用读卡器给 tf 卡存储照片，然后将 tf 卡插到 fpga 开发板中，然后 pc 上位机选择基础三部分，选择开始接收，上位机上显示着摄像头的残帧画面，是静止的状态，因为我们基础三与基础四是共用的 sdram，这样的话方便整合。之后我们按下开发板上的 key2 键，可以看到上位机刷新出 tf 卡中的图片，之后我们点击保存当前帧，就可以

将 tf 卡中的图片成功上传到 pc 端了。我们在这里在 pc 上位机上集成了抓包和转换图像的功能，实现了一键传图的便捷功能，这也是我们的一个创新点吧。

4.基础第四问：我们将 sw1 置为高电平，sw2 置为低电平，进入基础四部分。我们的基础四部分复用了米联客的 udp 打包逻辑，配合官方的例程，实现了高帧率传输视频至上位机，帧率高达 40+，也没有丢包效果。我们在上位机选择基础四部分，点击开始接收，之后上位机上便显示了 ov5640 摄像头采集的实时画面了，画面颜色清晰，流畅度高。

5.发挥第一问：这里我们将主板和从板连接在一起，之后将 sw1 与 sw2 均置为高位，切换到发挥第一问控制 led/数码管的功能，之后我们按下 key4，发送一条指令，可以看到从板 led 高两位亮，数码管显示数字 5，然后按下 key1 再发送一条指令，可以看到 led 第一位和第三位亮，数码管显示字母 b，与预期是完全一致的；之后我们将 sw1，sw2，sw3 均置为高位，切换到板间视频传输的功能，之后我们按下主板的 key1，可以看到，从板连接的显示屏上显示了主板 ov5640 实时采集的视频图像，并且画面很流畅，没有丢包效果。

6.发挥第二问：将 sw1 置高电平，sw2 置低电平，进入发挥第二问。上位机点击发挥二部分，开始接收就可以了。这里我们一共做了 15 种图像处理算法，依次是灰度化，二值化，反相，红色通道，重影，复古风，亮度调节，饱和度调节，sobel 边缘提取，素描效果，浮雕效果，腐蚀，膨胀，形态学边缘检测，通过 key4 键进行快速切换，通过 key3 键进行调节。

7.发挥第三问：这里我们主要是实现了肤色检测以及帧差法运动目标识别，下面给出相应的核心代码供参考。

肤色检测核心代码

```
`timescale 1ns / 1ps

module skin_color_algorithm

(

    input   wire          i_clk,      // 输入时钟

    input   wire          i_rst_n,    // 复位（低有效）

    input   wire          i_de,       // 数据有效

    input   wire [7:0]    i_r,        // 输入 R
```

```
    input   wire [7:0]    i_g,          // 输入 G
    input   wire [7:0]    i_b,          // 输入 B
    input   wire          i_hsyn,       // 行同步
    input   wire          i_vsyn,       // 场同步

    output  wire          o_hs,         // 输出行同步
    output  wire          o_vs,         // 输出场同步
    output  wire          o_de,         // 输出数据有效
    output  wire [7:0]    o_r,          // 输出 R（白=肤色，黑=非肤色）
    output  wire [7:0]    o_g,          // 输出 G
    output  wire [7:0]    o_b           // 输出 B
);

// YCbCr 转换核心公式
reg [15:0] y, cb, cr;  // 中间计算结果（16 位防溢出）
reg [7:0] cb_reg, cr_reg;  // 寄存转换后的 Cb/Cr
reg [2:0] sync_reg_h, sync_reg_v, sync_reg_de;  // 3 级同步（匹配计算延迟）

// 1. RGB→YCbCr 转换（单周期计算核心）
always@(posedge i_clk or negedge i_rst_n) begin
    if(!i_rst_n) begin
        y  <= 16'd0;
        cb <= 16'd0;
        cr <= 16'd0;
    end else begin
        // Y = (66R + 129G + 25B + 4096) >> 8
        y  <= 66*i_r + 129*i_g + 25*i_b + 16'd4096;
        // Cb = (-38R -74G + 112B + 32768) >> 8
        cb <= 112*i_b - 38*i_r -74*i_g + 16'd32768;
```

```
            // Cr = (112R -94G -18B + 32768) >> 8
            cr <= 112*i_r -94*i_g -18*i_b + 16'd32768;
        end
    end

    // 2. 移位得到 8 位 Cb/Cr（延迟 1 拍）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            cb_reg <= 8'd0;
            cr_reg <= 8'd0;
        end else begin
            cb_reg <= cb[15:8];
            cr_reg <= cr[15:8];
        end
    end

    // 3. 肤色阈值判断（Cb:77~127，Cr:133~173）
    reg [7:0] skin_r, skin_g, skin_b;
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            skin_r <= 8'd0;
            skin_g <= 8'd0;
            skin_b <= 8'd0;
        end else if(i_de) begin
            // 肤色→白色，非肤色→黑色
            if(cb_reg > 8'd77 && cb_reg < 8'd127 && cr_reg > 8'd133 && cr_reg < 8'd173)
begin
                skin_r <= 8'd255;
                skin_g <= 8'd255;
```

```
                skin_b <= 8'd255;
            end else begin
                skin_r <= 8'd0;
                skin_g <= 8'd0;
                skin_b <= 8'd0;
            end
        end else begin
            skin_r <= 8'd0;
            skin_g <= 8'd0;
            skin_b <= 8'd0;
        end
    end
    // 4. 同步信号打拍（匹配 3 级计算延迟）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            sync_reg_h <= 3'b000;
            sync_reg_v <= 3'b000;
            sync_reg_de <= 3'b000;
        end else begin
            sync_reg_h <= {sync_reg_h[1:0], i_hsyn};
            sync_reg_v <= {sync_reg_v[1:0], i_vsyn};
            sync_reg_de <= {sync_reg_de[1:0], i_de};
        end
    end

    // 输出赋值
    assign o_hs = sync_reg_h[2];
    assign o_vs = sync_reg_v[2];
    assign o_de = sync_reg_de[2];
```

```
    assign o_r = skin_r;
    assign o_g = skin_g;
    assign o_b = skin_b;

endmodule
```

帧差法多运动目标检测核心算法

```
`timescale 1ns / 1ps
帧差法目标检测（核心功能：两帧差分 + 阈值二值化）
module image_frame_diff_algorithm
(
    input   wire            i_clk,      // 输入时钟
    input   wire            i_rst_n,    // 复位（低有效）
    input   wire            i_hsyn,     // 行同步
    input   wire            i_vsyn,     // 场同步
    input   wire            i_de,       // 数据有效
    input   wire [23:0]     i_data_0,   // 当前帧（RGB888）
    input   wire [23:0]     i_data_1,   // 前一帧（RGB888）
    input   wire [15:0]     oprate16bit,// 阈值配置（bit15-8=0x02 时更新）
    output  reg             o_hs,       // 输出行同步
    output  reg             o_vs,       // 输出场同步
    output  reg             o_de,       // 输出数据有效
    output  reg [7:0]       o_data      // 输出二值化结果（白=运动目标）
);

// 核心参数：差分阈值（默认 30，支持配置）
reg [7:0] threshold = 8'd30;
always@(posedge i_clk or negedge i_rst_n) begin
    if(!i_rst_n) begin
```

```
            threshold <= 8'd30;
        end else begin
            // 配置命令：oprate16bit[15:8]=0x02 时，更新阈值为 bit7-0
            threshold <= (oprate16bit[15:8] == 8'h02) ? oprate16bit[7:0] : threshold;
        end
    end

    // 帧同步：等待 2 帧稳定后开始计算
    reg frame_valid;   // 帧有效标志
    reg [1:0] frame_cnt;   // 简化帧计数器（仅需 2 帧稳定）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            frame_cnt <= 2'd0;
            frame_valid <= 1'b0;
        end else if(i_vsyn == 1'b0 && i_vsyn_prev == 1'b1) begin   // 场同步下降沿（帧结束）
            frame_cnt <= frame_cnt + 2'd1;
            frame_valid <= (frame_cnt >= 2'd1) ? 1'b1 : 1'b0;   // 第 2 帧后开始有效
        end
    end

    reg i_hsyn_prev, i_vsyn_prev, i_de_prev;
    reg [7:0] data0_gray, data1_gray;
    // 1. 同步信号打拍 + 提取灰度值（取 G 通道近似灰度）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            i_hsyn_prev <= 1'b0;
            i_vsyn_prev <= 1'b0;
            i_de_prev <= 1'b0;
            data0_gray <= 8'd0;
```

```
            data1_gray <= 8'd0;
        end else begin
            i_hsyn_prev <= i_hsyn;
            i_vsyn_prev <= i_vsyn;
            i_de_prev <= i_de;
            data0_gray <= i_data_0[15:8];   // G 通道作为灰度值（简化，无需 RGB 转灰度）
            data1_gray <= i_data_1[15:8];
        end
    end

    // 2. 帧差计算 + 二值化（核心逻辑）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
            o_data <= 8'd0;
        end else if(frame_valid && i_de_prev) begin    // 帧有效且数据有效
            // 计算两帧灰度差的绝对值
            if(data0_gray >= data1_gray) begin
                o_data <= (data0_gray - data1_gray >= threshold) ? 8'd255 : 8'd0;
            end else begin
                o_data <= (data1_gray - data0_gray >= threshold) ? 8'd255 : 8'd0;
            end
        end else begin
            o_data <= 8'd0;
        end
    end

    // 3. 输出同步信号（打拍 1 级匹配延迟）
    always@(posedge i_clk or negedge i_rst_n) begin
        if(!i_rst_n) begin
```

```
            o_hs <= 1'b0;
            o_vs <= 1'b0;
            o_de <= 1'b0;
        end else begin
            o_hs <= i_hsyn_prev;
            o_vs <= i_vsyn_prev;
            o_de <= i_de_prev;
        end
    end

    endmodule
```

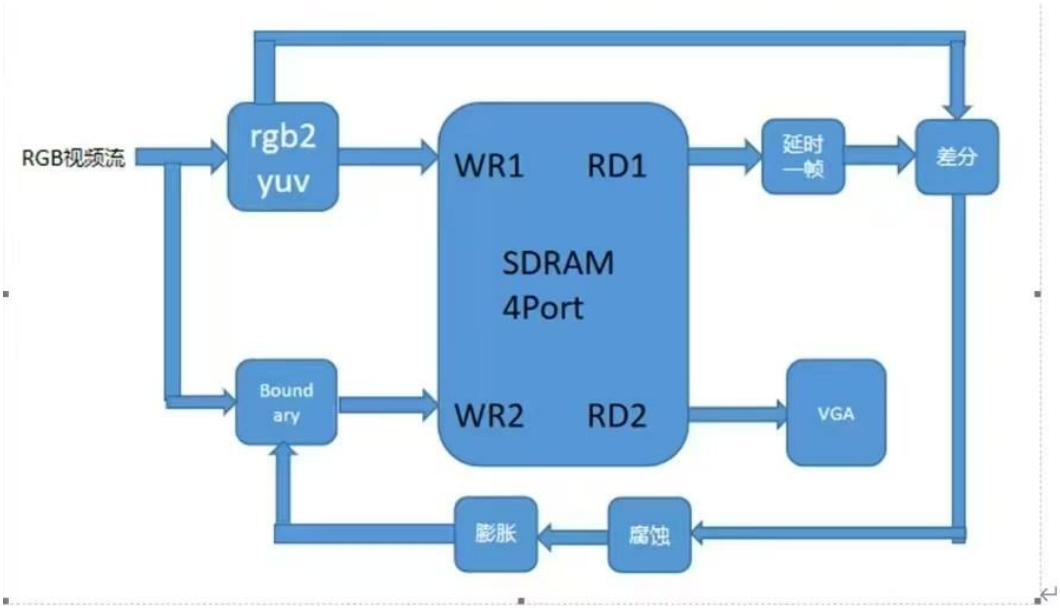


图 2.2 帧差法原理流程图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统硬件以 FPGA 核心开发板为控制中枢，构建 “核心控制 - 外设扩展 - 接口适配” 的三层架构。核心控制层集成 FPGA 主芯片及基础电路（时钟、复位），为全系统提供运算与控制支持；外设扩展层通过标准接口连接 OV5640 摄像头（图像采集）、TF 卡（数据存储）及其他辅助外设，实现数据输入与本地

存储；接口适配层包含信号转换、电平匹配电路，确保外设与核心芯片间的信号兼容。各层实现数据交互，硬件整体架构如 2.2 所示。

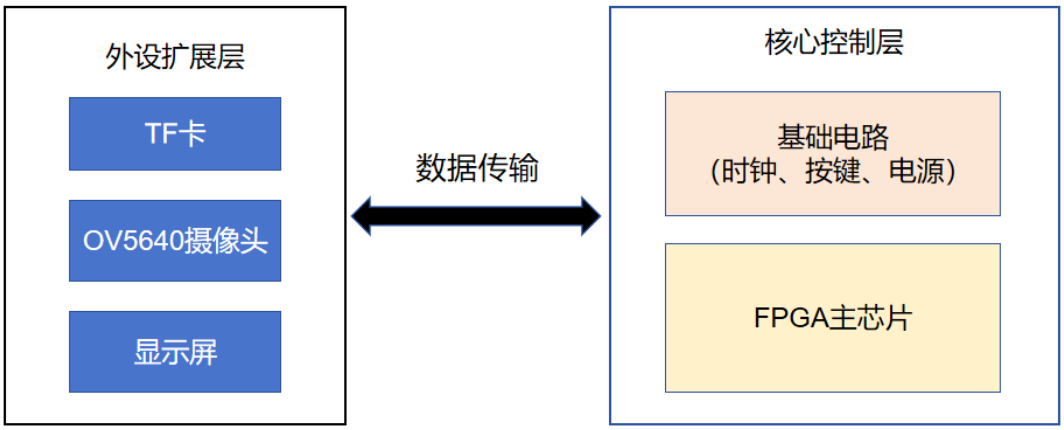


图 2.3 硬件整体架构

2.2.2 电路各模块介绍

（1）OV5640 驱动模块

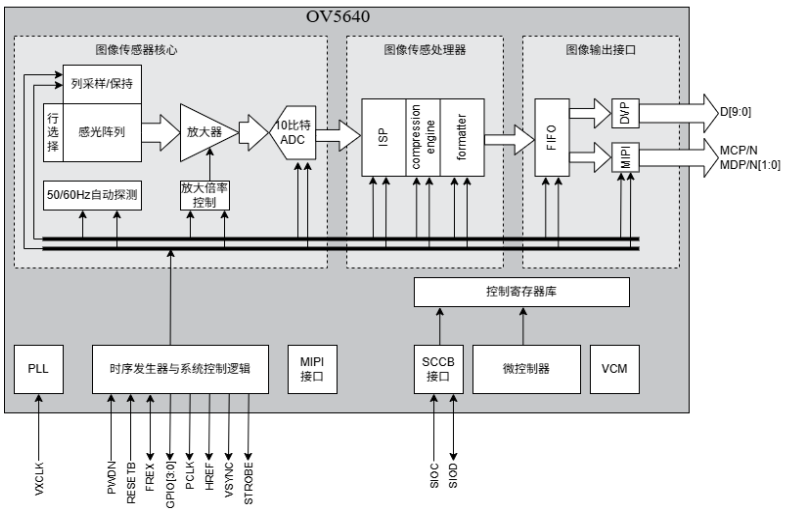


图 2.4 OV5640 功能框图

信号名	Bits	方向	功能描述
clk	1	Input	时钟。
rst_n	1	Input	复位信号，低电平有效。
cam_pclk	1	Input	数据像素时钟。
cam_vsync	1	Input	CMOS 场同步信号。
cam_href	1	Input	CMOS 行同步信号。

cam_data	8	Input	CMOS 数据。
cam_rst_n	1	Output	CMOS 复位信号，低电平有效
cam_pwdn	1	Output	CMOS 电源休眠模式选择信号
cam_scl	1	Output	CMOS SCCB_SCL 线
cam_sda	1	Inout	CMOS SCCB_SDA 线
cmos_h_pixel	13	Input	水平方向分辨率
cmos_v_pixel	13	Input	垂直方向分辨率
total_h_pixel	13	Input	水平总像素大小
total_v_pixel	13	Input	垂直总像素大小
capture_start	1	Input	图像采集开始信号
cam_init_done	1	Output	摄像头初始化完成
cmos_frame_vsync	1	Output	帧有效信号
cmos_frame_href	1	Output	行有效信号
cmos_frame_valid	1	Output	数据有效使能信号
cmos_frame_data	16	Output	有效数据

功能：实现 OV5640 摄像头的初始化配置与图像数据采集，模块顶层函数为 ov5640_driver_top，输入信号包括 FPGA 系统时钟、复位信号（高有效）等，输出信号为 8 位并行图像数据（img_data [7:0]）、数据有效信号（img_valid）及同步信号（href、vsync）。

（2）TF 卡控制模块

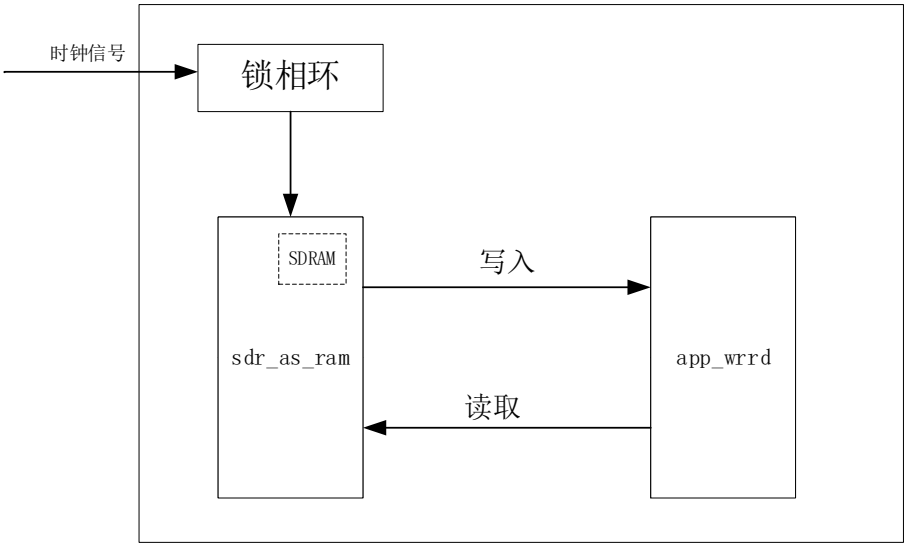


图 2.5 TF 卡控制架构图

信号名	Bits	方向	功能描述
clk_ref	1	Input	时钟信号
clk_ref_180deg	1	Input	时钟信号,与 clk_ref 相位相差 180 度
rst_n	1	Input	复位信号,低电平有效
sd_miso	1	Input	SD 卡 SPI 串行输入数据信号
sd_clk	1	Output	SD 卡 SPI 时钟信号
sd_cs	1	Output	SD 卡 SPI 片选信号
sd_mosi	1	Output	SD 卡 SPI 串行输出数据信号
wr_busy	1	Output	写数据忙信号
rd_start_en	1	Input	开始读 SD 卡数据信号
rd_sec_addr	32	Input	读数据扇区地址
rd_busy	1	Output	读数据忙信号
rd_val_en	1	Output	读数据有效信号
rd_val_data	16	Output	读数据
sd_init_done	1	Output	SD 卡初始化完成信号

功能：实现 TF 卡的初始化、数据读取与写入，模块顶层函数为 tf_spi_ctrl，输入信号包括时钟、复位信号、写使能（wr_en）、读使能（rd_en）及待写入数据

（wr_data [7:0]），输出信号为读出数据（rd_data [7:0]）、数据就绪（data_ready）及卡状态（card_status：0 - 未初始化，1 - 就绪，2 - 读写错误）。

（3）以太网 UDP 协议栈模块

UDP 格式：

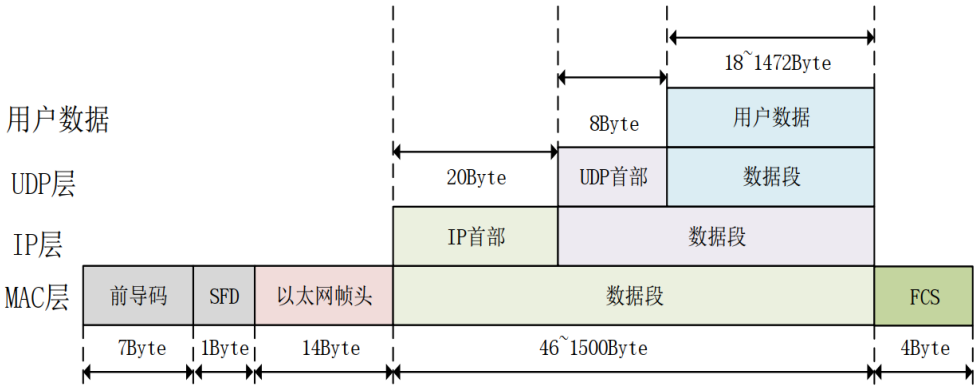


图 2.6 UDP 数据包格式

协议名称	位宽(Bytes)	功能描述
源端口号	2	16 位发送端端口号，用于区分不同服务的端口，端口号的范围从 0 到 65535。
目的端口号	2	16 位接收端端口号。
UDP 长度	2	16 位 UDP 长度，包含 UDP 首部长度+数据长度， 单位是字节（ byte）。
UDP 校验和	2	16 位 UDP 校验和。
用户数据	18~1472	用户数据。

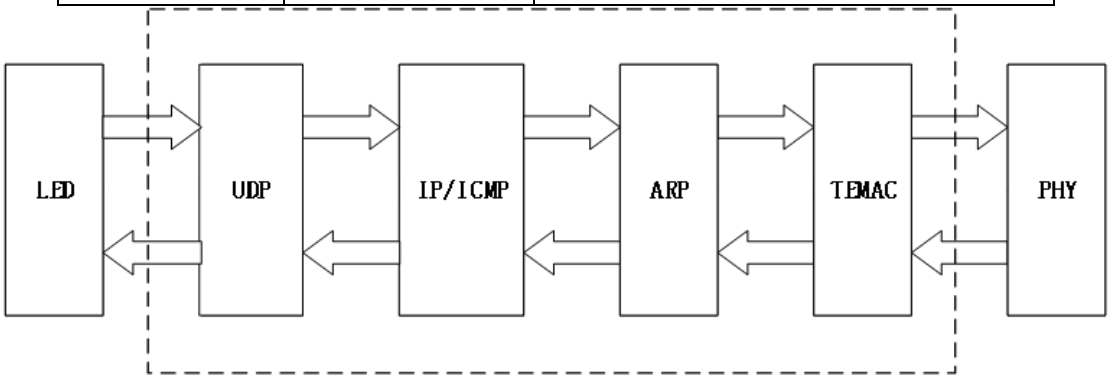


图 2.7 协议包传递结构

UDP 通用接口

信号名	Bits	方向	功能描述
udp_rx_clk	1	Input	UDP 接收时钟，千兆模式 125MHz，百兆模式 12.5MHz，十兆模式 1.25MHz。
udp_tx_clk	1	Input	UDP 发送时钟，千兆模式 125MHz，百兆模式 12.5MHz，十兆模式 1.25MHz。
reset	1	Input	全局复位信号，高电平有效。
udp2app_tx_ready	1	Output	UDP 和 APP 之间的握手信号，指示准备接收数据，高电平有效。
udp2app_tx_ack	1	Output	UDP 和 APP 之间的握手信号，指示响应接收数据，高电平有效。
app_tx_request	1	Input	UDP 和 APP 之间的握手信号，指示请求发送数据，高电平有效。
app_tx_data_valid	1	Input	APP 层发送数据有效指示信号，高电平有效。
app_tx_data	8	Input	APP 层发送数据。
app_tx_data_length	16	Input	APP 层发送数据长度，数据包的字节数。
app_rx_data_valid	1	Output	APP 层接收数据有效指示信号，高电平有效。
app_rx_data	8	Output	APP 层接收数据。
app_rx_data_length	16	Output	APP 层接收数据长度，数据包的字节数。
app_rx_port_num	16	Output	UDP 层的接收数据的源端口号。
app_tx_dst_port	16	Input	UDP 层的发送数据的目的端口号。
ip_tx_dst_address	32	Input	IP 层的 IP 地址。
temac_rx_valid	1	Input	MAC 层接收的数据有效信号，高电平有效。

temac_rx_data	8	Input	MAC 层接收的数据信号。
temac_tx_ready	1	Input	MAC 层接收数据的握手指示信号，低电平有效。
temac_tx_valid	1	Output	MAC 层发送数据的有效指示信号，低电平有效。
temac_tx_data	8	Output	MAC 层发送数据信号。
temac_tx_sof	1	Output	MAC 层发送数据的首字节指示信号。低电平有效。
temac_tx_eof	1	Output	MAC 层发送数据的末字节指示信号。低电平有效。
ip_rx_error	1	Output	IP 层的错误指示信号。
arp_request_no_reply_error	1	Output	ARP 层的请求无响应错误指示信号。

功能：实现以太网数据的封装与解析，基于安路 UDP 协议参考 IP 核开发，模块顶层函数为 udp_protocol_stack，输入信号包括系统时钟（100MHz）、复位信号、FPGA 发送数据（tx_data [7:0]）、上位机接收数据（rx_data [7:0]），输出信号为发送使能（tx_en）、接收就绪（rx_ready）及上位机指令（cmd [7:0]，如 0x01 - 启动采集，0x02 - 停止采集）。

（4）HDMI 显示时序模块

管脚编号	信号名称	功能描述
1	TMDS Data2+	TMDS 通道 2+，用于高速数据传输（视频、音频和辅助数据）
2	TMDS Data2 Shield	TMDS 通道 2 屏蔽接地
3	TMDS Data2-	TMDS 通道 2-
4	TMDS Data1+	TMDS 通道 1+用于高速数据传输
5	TMDS Data1 Shield	TMDS 通道 1 屏蔽接地
6	TMDS Data1-	TMDS 通道 1-

7	TMDS Data0+	TMDS 通道 0+用于高速数据传输
8	TMDS Data0 Shield	TMDS 通道 0 屏蔽接地
9	TMDS Data0-	TMDS 通道 0-
10	TMDS Clock+	TMDS 时钟正信号，提供数据同步时钟
11	TMDS Clock Shield	TMDS 时钟屏蔽接地
12	TMDS Clock-	TMDS 时钟-
13	CEC	消费电子控制，用于设备间控制信号
14	Reserved (HDMI 1.4+: Utility/HEAC+)	HDMI 1.4+中用于 HEAC（HDMI 以太网和音频回传通道）正信号，早期版本保留
15	SCL	DDC（显示数据通道）的 I2C 时钟信号用于 EDID 和 HDCP 通信
16	SDA	DDC 的 I2C 数据信号，与 SCL 配合传输显示器信息和加密数据
17	DDC/CEC Ground	DDC 和 CEC 的接地参考
18	+5V Power	提供 5V 电源
19	Hot Plug Detect (HPD)	热插拔检测信号，检测设备连接状态

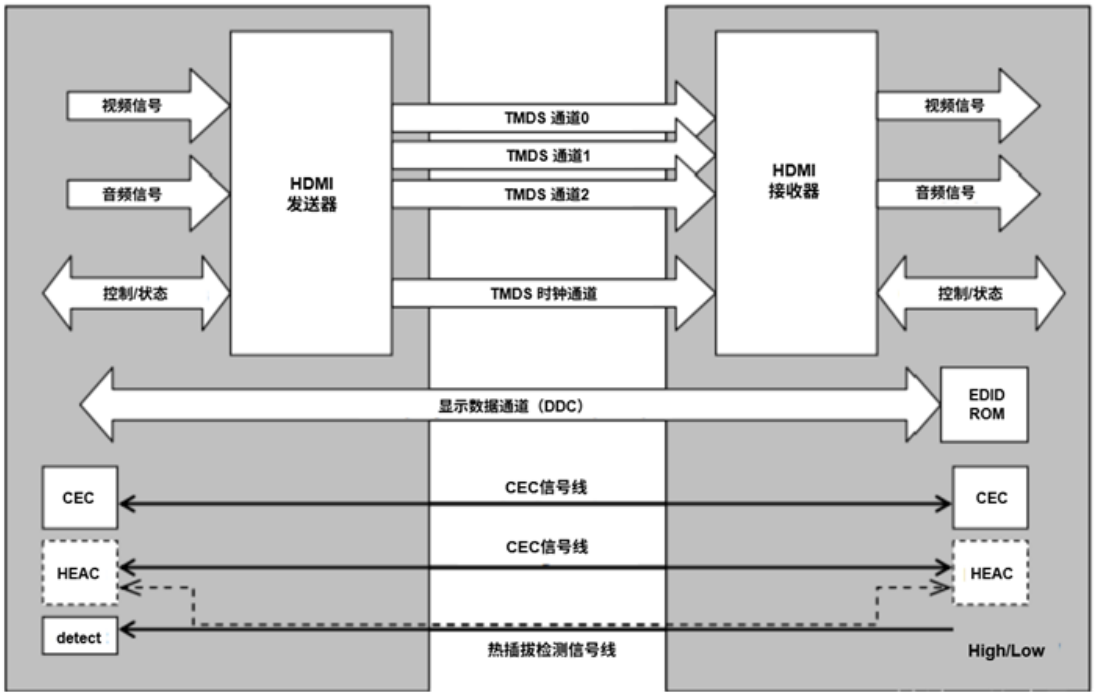


图 2.8 HDMI 系统架构

功能：生成 HDMI 显示所需的行场同步时序与图像数据驱动信号，模块顶层函数为 hdmi_timing_gen，输入信号包括系统时钟、复位信号、图像数据（img_data [23:0]，RGB888 格式），输出信号为 HDMI 行同步（hsync）、场同步（vsync）、数据使能（de）及 HDMI 差分信号（TMDS）。

2.3 软件系统介绍

2.3.1 软件整体介绍

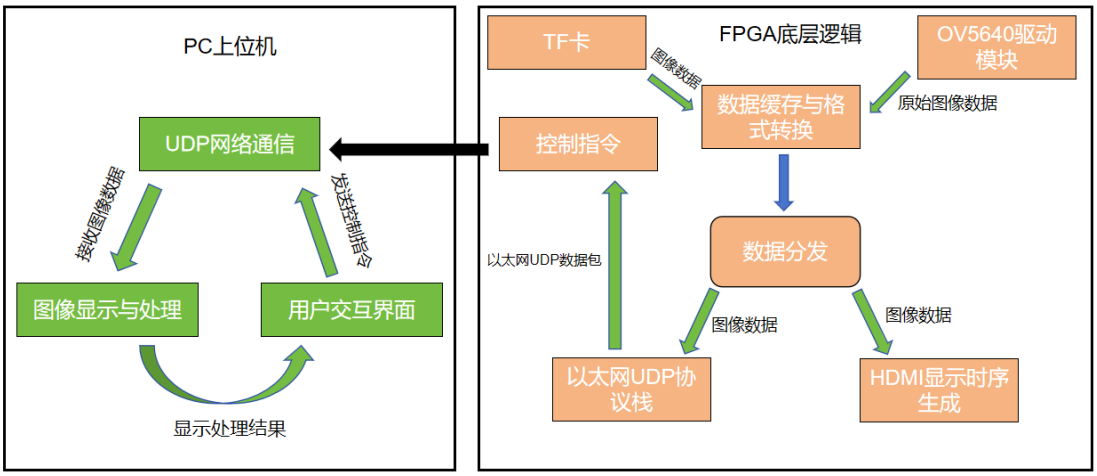


图 2.9 软件整体流程图

本系统软件采用 “FPGA 底层逻辑 +PC 上位机” 的双层架构，涵盖数据采集、协议处理、存储交互、显示控制及上位机监控五大核心功能，软件整体流程图如图 2.8 所示。

FPGA 底层逻辑：基于 Verilog HDL 语言开发，依托安路 FPGA 开发环境（TangDynasty）实现，集成 OV5640 驱动、TF 卡控制、以太网 UDP 协议栈、HDMI 显示时序生成四大核心模块，负责从硬件外设采集图像数据，完成数据缓存、协议封装后，输出至 HDMI 显示器与 PC 上位机，同时接收上位机指令控制外设工作；

PC 上位机：基于 python 语言开发，可实时将采集的图像数据显示出来，并可将所有基础与拓展功能无缝整合至统一平台，极大优化了用户体验，允许操作者便捷地切换显示模式、发送控制指令并即时查看处理结果。

2.3.2 软件各模块介绍

PC 上位机基于 Python 语言开发，构建 “一体化控制与显示平台”，无需复杂变量配置，聚焦功能便捷操作，核心模块设计如下：

（1）通信交互模块

核心功能：实现与 FPGA 的 UDP 数据收发，自动绑定预设 IP（如 PC 端 192.168.240.2、FPGA 端 192.168.240.1）与端口，无需手动配置。

（2）图像显示模块

核心功能：支持多场景图像实时渲染与模式切换，适配 FPGA 输出的原始图像（摄像头采集、TF 卡读取）及处理后图像。

（3）功能控制模块

核心功能：整合所有基础与扩展功能，提供可视化操作入口，无需手动输入代码或参数。

第三部分　完成情况及性能参数

3.1 整体介绍

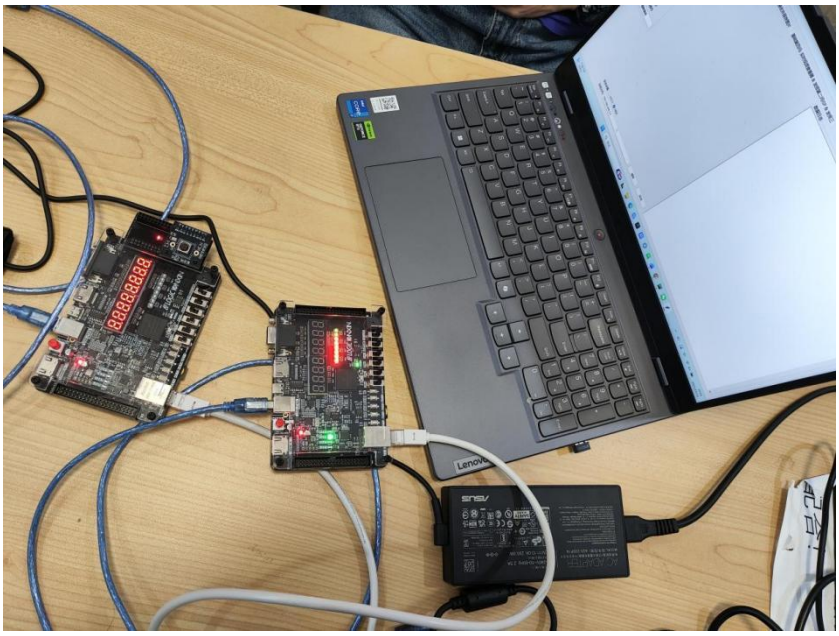


图 3.1　系统实物正面全局性照片

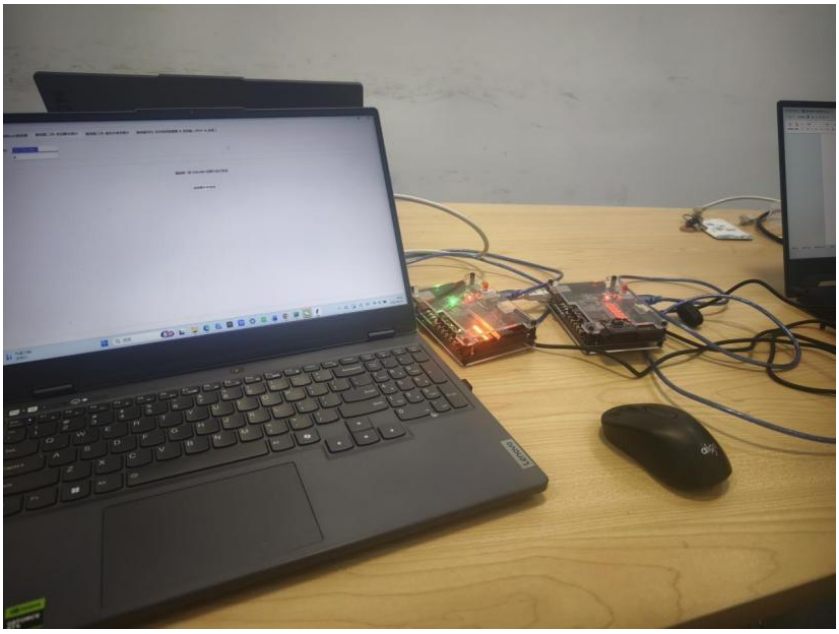


图 3.2　系统实物斜 45°全局性照片

3.2　工程成果/

3.2.1 机械成果；

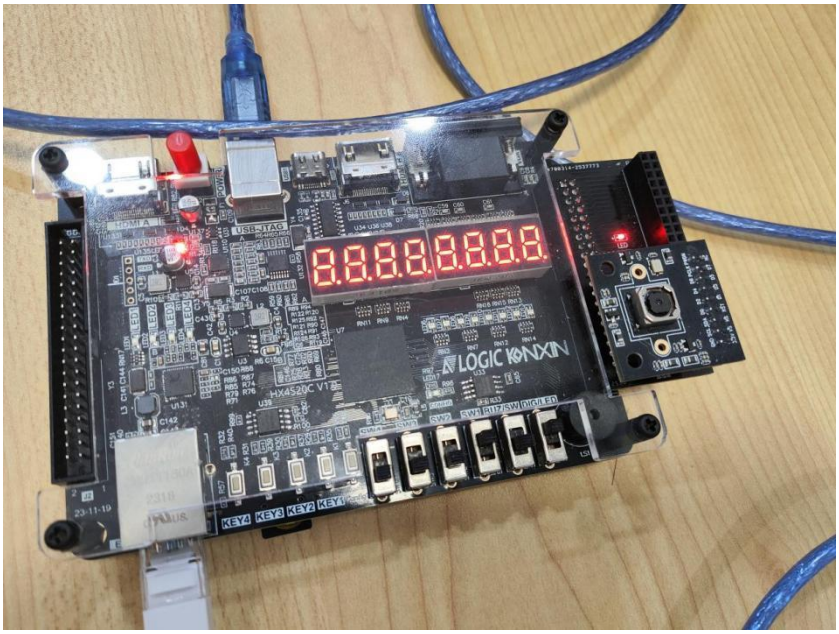


图 3.3　机械成果照片

3.2.2 软件成果；

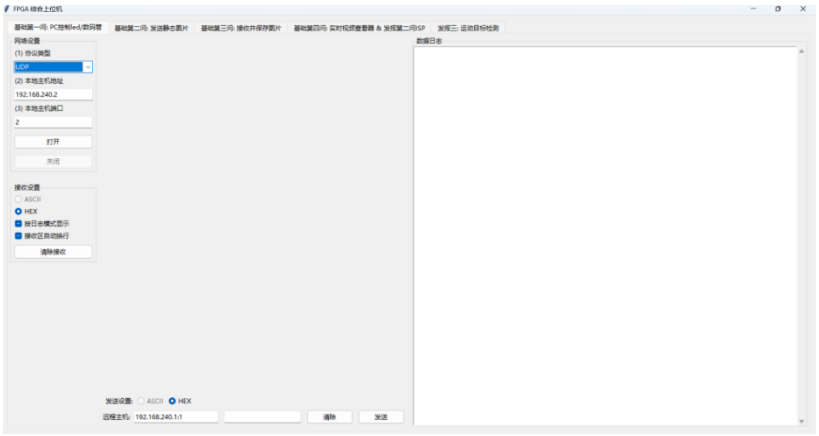


图 3.4　软件成果照片 1



图 3.5　软件成果照片 2

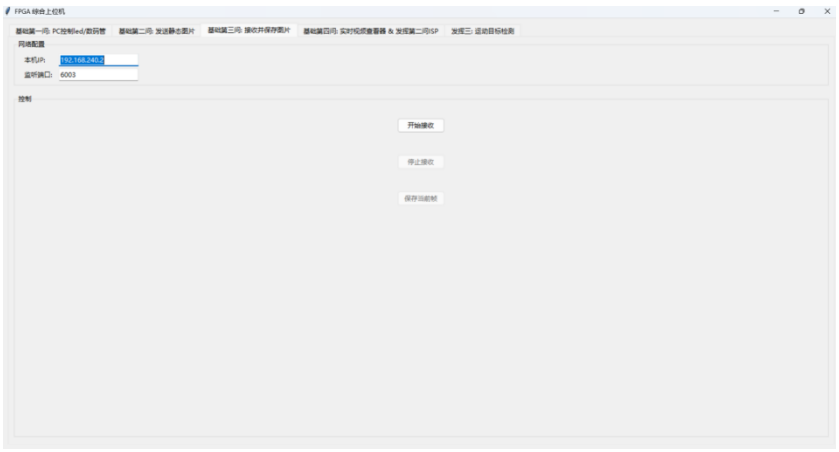


图 3.6　软件成果照片 3



图 3.7　软件成果照片 4

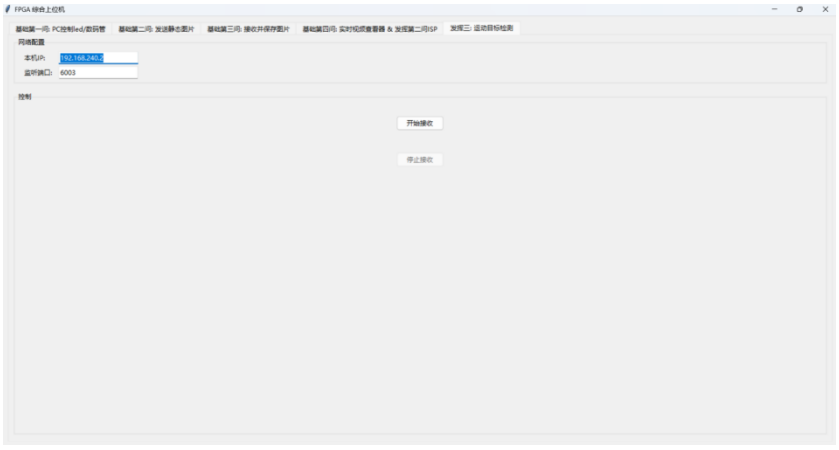
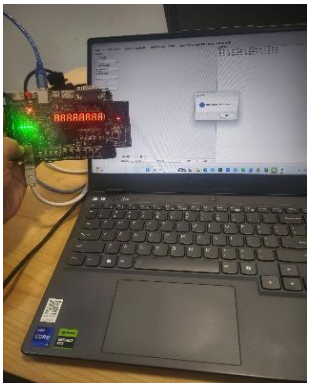
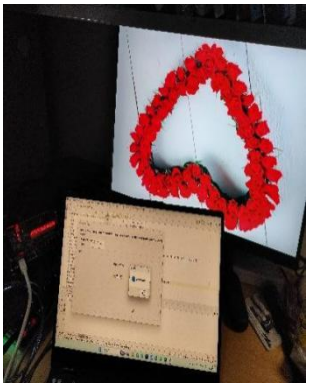
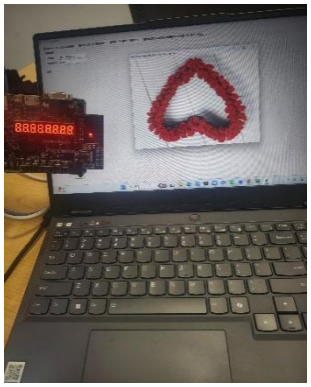
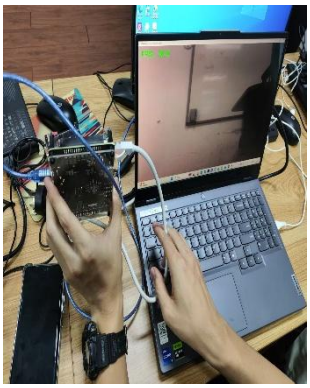
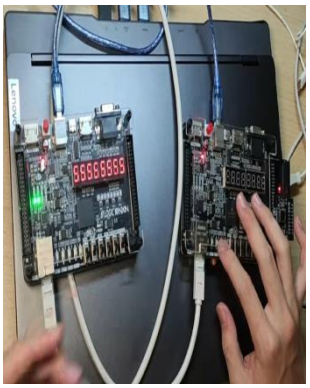
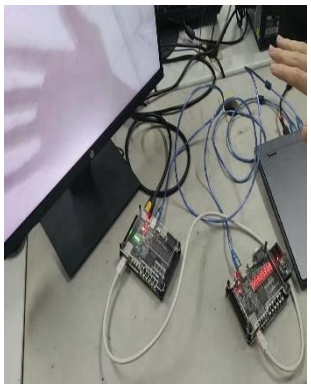
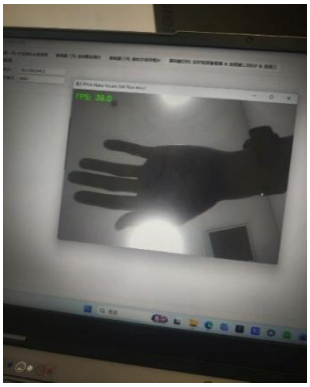
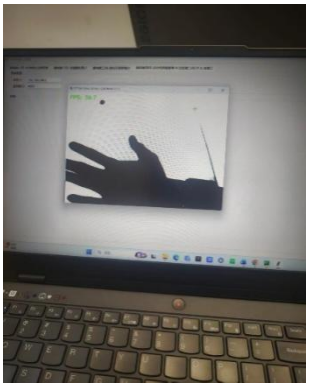
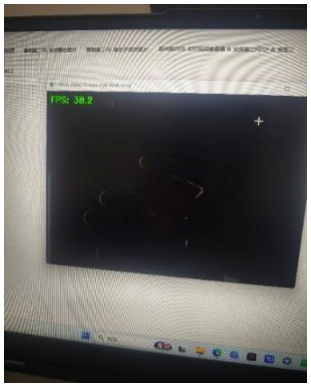
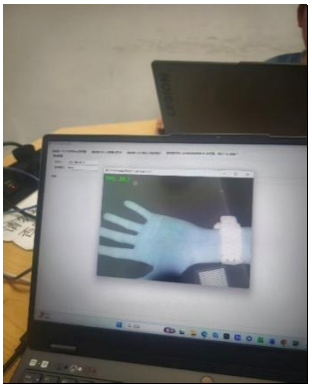
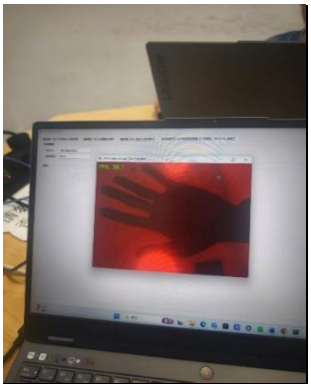
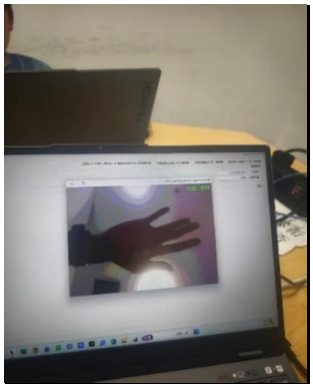
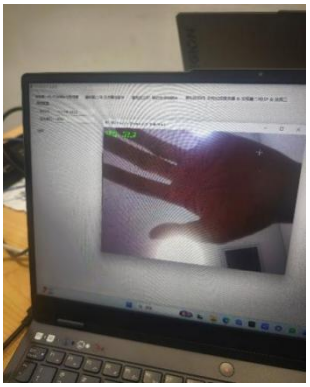
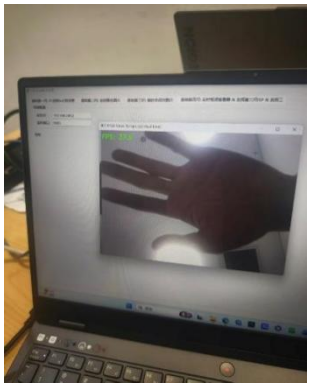
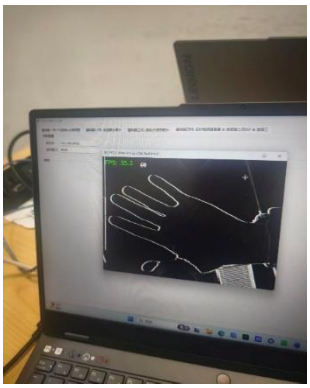
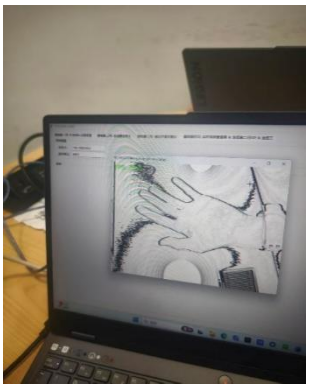
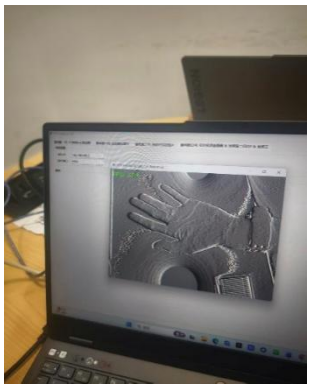


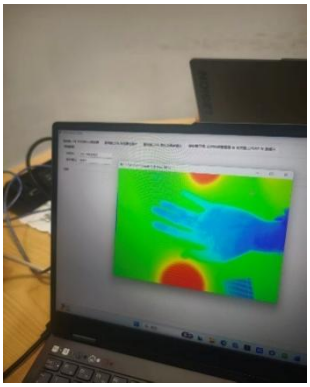
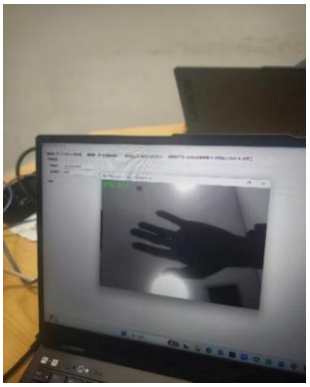
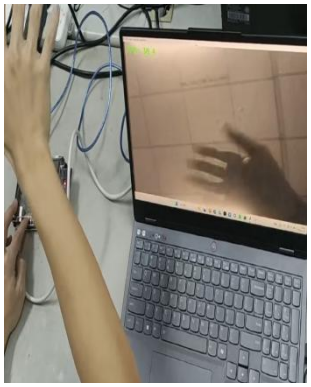
图 3.8　软件成果照片 5

3.3 特性成果

		
基础第一问： 使用以太网接收 PC 上简单命令，控制开发板上 LED 灯、数码管。	基础第二问： 使用以太网接收 PC 上位机的图片，控制开发板 HDMI 显示。	基础第三问： 使用以太网发送开发板上图片（存储于 TF 卡上）至 PC 上位机，并在上位机上显示。
		
基础第四问： 使用开发板上外接摄像头模块作为视频输入，并通过以太网传输至 PC 上，进行显示。	扩展第一问（1）： 两块开发板做以太网连接，实现板间控制 LED 和数码管。	扩展第一问（2）： 两块开发板做以太网连接，实现板间视频传输。
扩展第二问： FPGA 端视频图像实时处理，在 PC 端显示处理结果。		

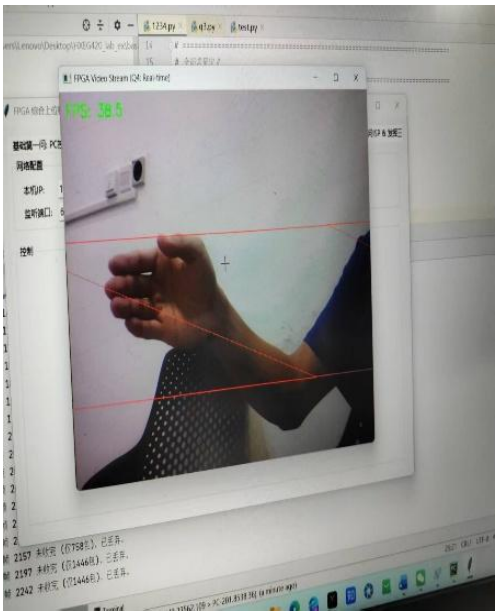
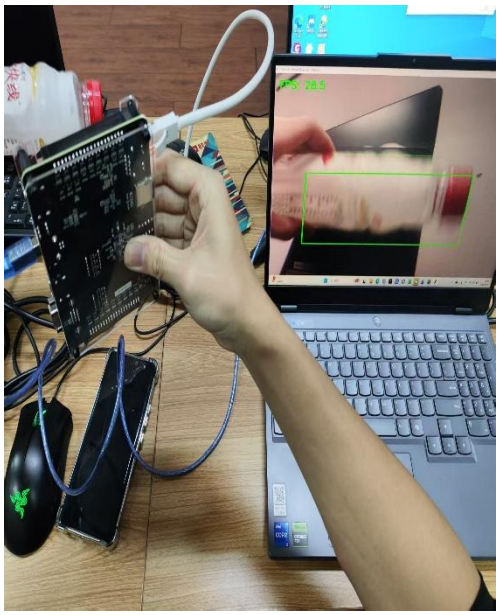
		
1、**灰度化** 将彩色图像转为灰度图的处理，通过让像素 RGB 值趋同，使图像呈现从黑到白的不同灰度等级。	2、**二值化** 将图像像素转为黑、白两种数值的处理，通过阈值判定，简化图像以突出关键目标。	3、**形态学边缘提取** 借膨胀、腐蚀等形态学运算，凸显图像灰度突变区域，以此获取图像边缘。
		
3、**反相** 将图像各像素颜色转为对应补色，比如黑变白、白变黑，呈现与原图相反的色彩效果。	4、**红色通道** RGB 色彩模式中负责承载图像红色信息的通道，仅反映红色分量分布。	5、**重影** 设置图像中同一物体出现重复虚影。

		
6、**复古风** 模仿过去特定年代风格的设计潮流，在图像中通过调低饱和度、加暖色调等营造怀旧质感。	7、**饱和度调节** 改变图像色彩的鲜艳程度，调高更艳丽，调低则色彩更淡甚至趋近灰度。	8、**亮度调节** 改变图像像素明暗程度，调高画面更亮，调低则画面更暗，以此适配视觉需求。
		
9、**Sobel 边缘提取** 用 Sobel 算子计算图像灰度梯度，检测水平和垂直方向灰度突变，进而提取图像边缘。	11、**素描** 通过强化边缘、降低色彩等方式，将图像转化为类似铅笔勾勒线条质感的效果。	12、**浮雕** 通过凸显像素灰度差异模拟立体浮雕效果，经算法处理让图像呈现凹凸的纹理质感。

		
13、**热力图** 用不同色调表示数据强度，直观呈现数据分布差异，用来展示热点区域等信息。	14、**腐蚀** 通过结构元素侵蚀图像亮区，缩小亮部区域、消除细小噪点与断开细连接。	15、**膨胀** 借结构元素扩张图像亮区，扩大亮部区域、填补细小缺口并连接断开的相近区域。

扩展第三问：

对开发板上采集的视频图像中的物体进行目标检测和识别，并通过以太网传输至上位机后显示目标检果。

	
1、**肤色检测：** 在视频中定位并提取人体肤色区域，依据肤色色彩特征判定，可用于人	2、**多目标运动识别：** 检测视频中多个运动目标，再使用方框将其锁定，可用于监控、自动驾驶

脸检测、人像处理等场景。	等场景。

第四部分 总结

4.1 可扩展之处

本系统在现有基础上仍有广阔的扩展空间。首先，在硬件层面，可增加更多传感器（如多路摄像头、麦克风阵列）以实现多模态数据融合传输与处理。其次，在算法层面，可将当前检测算法升级为更复杂的基于 CNN 的轻量化目标识别模型，或集成图像压缩（如 JPEG2000）模块以极大降低网络带宽占用。再者，在网络协议层面，可扩展为多板卡级联的分布式处理系统，通过引入 UDP 或精确时间协议（PTP）实现多节点同步，适用于更复杂的工业控制场景。最后，在应用层面，可开发配套的移动端 APP 或云平台接口，实现远程监控与控制，构建完整的物联网解决方案。

4.2 心得体会

从赛题发布到完成这个集以太网通信、视频流处理与高级视觉功能于一体的 FPGA 系统，这段研发历程是一次极具价值的全面工程实践，挑战与收获并存。

1. 最大的挑战源于系统集成。

单个功能的实现并非最困难之处，真正的难点在于将众多时序复杂的模块无缝整合为一个稳定协同的整体。我们采取分模块策略，先后攻克了 MAC 层与 IP 协议栈、摄像头采集与显示流水线。然而，当连接网络接收与视频显示模块时，异步突发的网络数据与严格时序的视频流直接冲突，导致画面撕裂。通过深入分析，我们创新性地设计了“异步 FIFO+帧缓存管理”的双缓冲机制：网络端将完整帧数据存入 SDRAM 的“后台”缓存，显示端在需要时通过帧同步信号切换至已写满的缓存作为“前台”进行读取，网络端则写入另一块缓存。这种“乒乓操作”有效解决了跨时钟域同步问题，确保了画面流畅稳定。这一过程让我们深刻认识到，在 FPGA 系统设计中，数据流架构和时序同步的重要性甚至超过算法逻辑本身。

2. 对“并行流水线”的深入理解是另一大飞跃。

在实现 15 种图像处理算法时，我们最初沿用软件思维，采用大型状态机进

行模式选择，导致资源占用高、频率受限。后来我们彻底转向 FPGA 的并行流水线理念：将色彩空间转换、灰度化等预处理设为常驻流水线级，将边缘检测、形态学处理等算法设计为可旁路的并行单元。图像数据流经预处理后，根据模式选择流入特定算法单元，最后输出。这种设计使算法物理并行，大幅提升了数据吞吐率和系统时钟频率，实现了真正的实时处理。这一从“顺序思维”到“并行思维”的转变，是技术认知上的关键突破。

3.软硬件协同调试的艺术至关重要。

例如，在运动目标检测中，检测框偶尔漂移。排查过程极为曲折，最终借助嵌入式逻辑分析仪抓取内部信号，并与上位机数据包逐字节对比，发现根源是图像行列计数器同步时机存在细微偏差。这一经历警示我们，复杂系统的问题往往隐藏在意想不到的接口细节中，必须善于利用调试工具，建立端到端的验证思维，才能高效定位问题。总而言之，本项目不仅使我们系统掌握了 FPGA 复杂系统设计方法，更极大地锻炼了工程规划、团队协作与解决实际问题的能力。最终系统稳定运行的成就感，为未来的技术生涯奠定了坚实的信心基础。

第五部分 参考文献

[1] 安路科技. EG4 系列 FPGA 产品数据手册[Z].

[2] 胡振波. FPGA 时序约束与分析[M]. 北京: 清华大学出版社, 2020.

[3] 夏宇闻. Verilog 数字系统设计教程[M]. 北京: 北京航空航天大学出版社, 2017.

[4] 王诚, 吴继华, 等. FPGA 设计实战演练(逻辑篇)[M]. 北京: 人民邮电出版社, 2015.

[5] 吴继华, 王诚. 设计与验证: Verilog HDL[M]. 北京: 人民邮电出版社, 2019.

[6] 何宾. 以太网技术与应用[M]. 北京: 电子工业出版社, 2021.

[7] 冈萨雷斯. 数字图像处理(第四版)[M]. 阮秋琦, 等译. 北京: 电子工业出版社, 2020.

[8] 王耀南, 李树涛. 机器视觉与图像处理[M]. 北京: 科学出版社, 2018. [9] 刘波. 基于 FPGA 的嵌入式图像处理系统设计[J]. 电子技术应用, 2022, 48(5): 12-16.

[10] 张峰, 李静. 千兆以太网 MAC 控制器的 FPGA 设计与实现[J]. 单片机与嵌入式系统应用, 2021, 21(7): 25-28.

[11] 周维, 刘凯. 基于 Sobel 算子的实时图像边缘检测 FPGA 实现[J]. 现代电子技术, 2023, 46(2): 45-49.

[12] Xilinx Inc. AXI4-Stream Infrastructure IP Suite[EB/OL]. (2022) [2024-10-30]. https://www.xilinx.com.